



CEN429 Dönem Projesi

Proje Rehberi — 2026-2027 Güz

Dr. Öğr. Üyesi Uğur CORUH

Projenin amacı

Bu projede, seçtiğiniz bir uygulamayı **sertifikasyon sürecinden geçiyormuş gibi** tasarlayıp geliştireceksiniz:

- bir **C++ konsol uygulaması**
- onun güvenliğini sağlayan bir **dinamik kütüphane (DLL/.so)**

Amaç: dönem boyunca öğrendiğiniz güvenlik yöntemlerini **tek bir üründe** birleştirmek ve hepsini bir **güvenlik kılavuzuyla** (S0–S17) belgelemek.

Kısaca

- **Takım:** en çok 5 kişi (tek başınıza da olur); bir konuyu yalnız bir takım alır; 3. haftadan (04.10.2026) sonra takım değişmez
- **Dil/araçlar:** C++ (DLL/.so), CMake/CTest, SQLite, SoftHSM/PKCS#11, OpenSSL
- **İki teslim:** Vize (RAP1) — ürünün ilk yarısı; Final (RAP2) — tam ürün + test **sonuçları**
- **Teslimin kalbi:** kodun yanında, sertifikasyon belgesi gibi yazılmış **güvenlik kılavuzu** (S0–S17)
- Bütün veriler **sentetiktir**; depoda gerçek sır, anahtar ya da kişisel veri bulunmaz
- Ayrıntılı puanlama anahtarı ve kanıt şablonları Microsoft Teams'te paylaşılabilir; **bu sayfadaki kriterler ve puanlar bağlayıcıdır**

Kontrol noktaları

Vize kontrolü	7. hafta (30.10.2026) · RAP1 — ürünün ilk yarısı
Final kontrolü	15. hafta (25.12.2026) · RAP2 — tam ürün + test sonuçları

1. Takvim

Hafta	Tarih	Etkinlik
3. hafta sonu	04.10.2026	Konu ve takım seçiminin son günü
4. hafta	09.10.2026	Proje planının ders sorumlusuna onaylatılması
7. hafta	30.10.2026	Vize gösterimi + ara rapor (RAP1)
8. hafta	31.10–08.11.2026	Quiz-1 (1–6. haftalar)
15. hafta	25.12.2026	Final gösterimi + final raporu (RAP2)
16. hafta	04–17.01.2027	Quiz-2 (9–14. haftalar)

Haftalık ders içeriğinin tamamı için bkz. izlence. Tarihler değişirse duyuru ders sınıfından yapılır.

2. Değerlendirme yapısı — ağırlıklar

- $Vize = 0,6 \cdot RAP1 + 0,4 \cdot Quiz-1$
- $Final = 0,7 \cdot RAP2 + 0,3 \cdot Quiz-2$
- $Başarı\ notu = 0,4 \cdot Vize + 0,6 \cdot Final$

RAP1 (7. hafta): ürünün ilk yarısı (S2–S5 + temel korumalar)

RAP2 (15. hafta): tam ürün + S16 sonuçları

Öğrenme çıktıları (1/2)

ÖÇ	Tanım
ÖÇ.1	Yaygın yazılım güvenlik açıklarını (taşma, enjeksiyon, bellek sızıntısı) tanımlar ve sınıflandırır
ÖÇ.2	Temel şifreleme yöntemlerini ve güvenli iletişim ilkelerini (SSL/TLS) açıklar
ÖÇ.3	Kod sağlamlaştırma tekniklerini (girdi doğrulama, güvenli bellek, RASP, kod gizleme) açıklar ve uygular
ÖÇ.4	Şifreleme ve kimlik doğrulama ile güvenli iletişim kanalları kurma ilkelerini açıklar

Öğrenme çıktıları (2/2)

ÖÇ	Tanım
ÖÇ.5	Güvenli yazılım tasarımı ilkeleriyle (en az ayrıcalık, savunma derinliği) bir koruma planı oluşturur
ÖÇ.6	Yazılım güvenlik açıklarını tespit etmek için temel güvenlik incelemesi ve açık değerlendirmesi yapar
ÖÇ.7	Güvenli programlama standartlarını (ETSI, EMV, FIPS) ve sızma testi planlaması ilkelerini bilir

Her iki kontrol de bütün öğrenme çıktılarına dokunur; RAP1 temel/erken düzeyi, RAP2 ileri düzeyi ve tam ürünü ölçer.

3. Araçlar ve kurulum (1/2)

Araç	Ne için	Çıktı
C++ derleyicisi (GCC/Clang/MSVC)	Konsol uygulaması + DLL/.so derlemesi	Çalışan binary
CMake + CTest	Derleme sistemi + birim testi	Test raporu
SQLite	Beklemedeki veri deposu	.db şeması
SoftHSM / PKCS#11	Anahtar sarma/saklama benzetimi	Anahtar deposu yapılandırması

3. Araçlar ve kurulum (2/2)

Araç	Ne için	Çıktı
OpenSSL	Kriptografi, TLS, imzalama	Kütüphane bağlantısı + sertifikalar
Doxygen	Kod belgelendirmesi	PDF/HTML çıktısı
Git / GitHub	Sürüm kontrolü, iş birliği	Commit geçmişi, çekme isteği (PR)
GitHub Actions	Sürekli entegrasyon (CI)	Yeşil derleme/test rozeti

Şablon deposu

Şablon: `ucoruh/cpp-cmake-ctest-template`

- **Fork etmeyin:** herkese açık bir deponun fork'u özel yapılamaz
- Şablon sayfasında **Use this template** → **Create a new repository**; depo adı `cen429-proje-ad-soyad-cpp` , **Private**
- Ders sorumlusunu (`ucoruh`) ve varsa takım arkadaşınızı **işbirlikçi (collaborator)** olarak ekleyin

!!! tip "Şablonu tam kullanın"

Şablon; derleme, CTest ile birim testi, dokümantasyon üretimi, test/dokümantasyon kapsamı ölçümü ve

paketlemeyi sağlar. Şablon standartlarına uymayan teslimler kabul edilmez. Sürüm (release) üretin; sürüm

kimliği ve özet değerlerini kaydedin.

Projenizi yerelde gösterme

GitHub Free'de özel (private) bir depoda GitHub Pages sitesi açılmaz. Her şeyi **kendi bilgisayarınızda** gösterirsiniz:

- `7-build-all-windows.bat` (Linux/WSL: `7-build-all-linux.sh`) derler, test eder, bütün raporları üretir
- `9-open-site-windows.bat` (Linux: `9-open-site-linux.sh`) tam siteyi `http://localhost` adresinde açar: bütün raporlar (testler, kod kapsamı, dokümantasyon kapsamı; Windows ve Linux) ve API belgeleri
- `release/` bütün çıktıları tutar: uygulama/exe, kütüphane, raporlar, API belgeleri, `site.zip`, kaynak kod, `ASSETS.md`, `SHA256SUMS.txt`
- `10-release-windows.bat` (Linux: `10-release-linux.sh`) GitHub Release'i oluşturur; sürümler özel depolarda da çalışır

Gösterim kontrol listesi

Gösterimde sırayla neyi açıp göstereceğiniz:

1. Yerel sitenin **ana sayfası** (`9-open-site-...` , `http://localhost` adresinde).
2. **Her rapor sayfası**: testler, kod kapsamı, dokümantasyon kapsamı (Windows ve Linux).
3. **API belgeleri**.
4. `release/` klasörünün içeriği.
5. Uygulamayı **release arşivinden çalıştırma**.
6. Özel deponuzun **GitHub Release sayfası** (ders sorumlusu collaborator olarak ekli).

Proje düzeni

- `lib/` — güvenlik kütüphanesi (DLL/.so): kriptografi, RASP, kod gizleme, veri koruma fonksiyonları
- `app/` — konsol uygulaması: kütüphaneyi kullanan kullanıcı arayüzü
- `test/` — CTest birim testleri (kripto ve koruma fonksiyonları)

Geliştirme ortamı kurulumu ve ön bilgi için bkz. **Ön gereksinimler** sayfası.

4. Konu seçimi

Proje konuları rehber sayfasının sonundaki **Ek — Proje konuları listesi** bölümündedir (**100 konu**).

Her konu kutusunda:

- Uygulamanın kısa tanımı
- Temel özellikleri
- **Korunacak varlıklar**
- O konuda **öne çıkan güvenlik gereksinimleri**

Nasıl seçilir?

1. Rehber sayfasının sonundaki listeyi inceleyin ve bir konu seçin
2. Seçiminizi paylaşılan **takım ve konu listesine** yazın (bağlantı Teams duyurusunda) — **bir konuyu yalnız bir takım alabilir**, tabloya ilk yazan takım alır
3. Proje planınızla birlikte ders sorumlusuna onaylatın; **onaydan sonra konu değiştirilmez**

Konu seçiminde kurallar

- v. bölümdeki 15 gereksinimin **hepsi** her projede geçerlidir; konu kutusundaki "öne çıkan gereksinimler" o konuda en doğal ve en çok puan getiren uygulama yerlerini gösterir
- Listede olmayan bir fikriniz varsa, ders sorumlusunun onayıyla seçebilirsiniz
- Bu dersi tekrar alıyorsanız **önceki projenizden farklı** bir konu seçin

5. Gereksinimler

15 gereksinim · vize ve final kapsamına ayrılmış

Her maddeyi ya karşılayın ya da gerekçeli olarak devredin/kapsam dışı bırakın (13. hafta, S14)

5.1 Vize kapsamı (RAP1)

4.1 Geliştirme ortamı güvenliği

Hafta 1, 12 · S13

- Yazılım geliştirme akışı ve **değişiklik yönetimi** (temel çizgi → talep → sınıflandırma → onay → yayın → doğrulama)
- **Git** ile sürüm kontrolü; erişim kaydı; imzalı sürüm
- Geliştirme bilgisayar/sunucu güvenliği (kısa politika)

4.2 Kullanımdaki veri güvenliği

Hafta 3, 6 · S7

- Bellekteki hassas veri kullanımdan sonra **güvenli silinir** (`memset_s` benzeri)
- Çalışma zamanı veri koruması (gölge kopya/bütünlük — 6. hafta)

4.3 Aktarımdaki veri güvenliği

Hafta 3, 10 · S6, S11

- **TLS 1.3**; sertifika zinciri + **SAN** doğrulaması; **sabitleme (pinning)** + yedek pin
- Şifrelenmiş **oturum anahtarı**; **cihaz/sürüm bağlama**; bütünlük + kimlik doğrulama; sunucu doğrulama kodu

4.4 Beklemedeki veri güvenliği

Hafta 3, 10, 11 · S8

- Dosya/DB şifreleme **AEAD** (AES-GCM); nonce tekrarsız
- Hassas anahtarlar için **whitebox** ya da **SoftHSM/PKCS#11**; kararın **gerekçesi** (11. hafta)

4.8 Arayüz tanımları ve korunması

Hafta 1, 4 · S3, S6

- Tüm arayüzler erişim kontrolü + kimlik doğrulama ile korunur
- Girdi **güven sınırında** doğrulanır

4.9 Kod sağlamlaştırma

Hafta 4, 9, 14 · S9

- Opak döngü/yüklem, ad/dosya/dize/aritmetik gizleme, opak boolean, sahte işlem/ölü dal
- **Kontrol akışı düzleştirme** + rastgele çıkış, sürümde **log kapalı**
- Maliyeti **ölçün** (9/14. hafta demoları)

4.10 RASP

Hafta 6 · S10

- Checksum bütünlük denetimi, çağırılan uygulama hash/imza doğrulaması
- Kök/emülatör tespiti, **hook/anti-debug**
- Kurcalama (tamper) tespiti + yanıt, kontrol akışı sayacı

4.11 Bellek koruması

Hafta 4 · S9

- Derleyici/OS korumaları (yığın koruyucu, PIE, RELRO, NX, CFI)
- Hassas veri kullanım sonrası temizlenir

5.2 Final kapsamı (RAP2)

4.5 Statik varlıkların korunması

Hafta 1, 4, 9 · S5, S9

Gizli anahtarlar, özet değerleri, kaynak kodu, kaynaklar: **şifreleme + erişim kontrolü + gizleme**

4.6 Dinamik varlıkların korunması

Hafta 3, 6 · S5, S8

Cihaz/uygulama parmak izleri, oturum verileri, dinamik anahtarlar **şifrelenir**

4.7 Varlık yönetimi

Hafta 1, 3, 13 · S5

Her varlık için: **ad, açıklama, konum (tablo/sütun), kaynak, boyut, oluşturma/silme zamanı, varsayılan, koruma şeması (C/I/I+)**

4.12 Kriptografi ve sertifikalar

Hafta 3, 10 · S8, S11

- Doğru algoritma/kip/dolgu
- İmza doğrulaması (==1 , sürüm kapsamı)
- SSL/TLS + pinning + karşılıklı kimlik doğrulama

4.13 Sertifikasyon ve sızma testi planı

Hafta 12, 13 · S16, S14, S17

- Standart eşlemesi (ETSI/EMVCo/GSMA/PCI/MASVS)
- **Sızma testi planı** (kapsam, kurallar, yöntem, test kartı) ve **sonuçları**
- Saldırı potansiyeli + CVSS

4.14 Binary uygulama koruması

Hafta 6, 9, 11, 14 · S9, S10, S15

- **Tespit** (checksum, anti-debug, emülatör)
- **Savunma** (gizleme, dize/kaynak şifreleme, çağrı gizleme)
- **Caydırma** (yanıt/kapanma politikası)

4.15 OWASP ve derleme/dağıtım hattı

Hafta 5, 14 · S13, S15

- OWASP MASVS/ASVS ilkeleri
- **SBOM** (CycloneDX) + bağımlılık taraması
- Gizleme + imzalama içeren derleme hattı

5.3 Her iki kontrolde ortak kurallar

- Değerler **sentetik** olmalı; depoda gerçek sır, anahtar ya da kişisel veri **bulunmaz**
- Her gereksinim maddesi kod içinde **görülebilir** olmalı ve güvenlik kılavuzunun ilgili S bölümünde **belgelenmelidir**; belgelenmeyen bir uygulama karşılanmamış sayılır
- Bir gereksinim karşılanamıyorsa **gerekçesiyle** S14'e yazılır; sessizce atlanamaz
- Uygulama yalnız kendi bilgisayarınızda (localhost) çalışır; başka bir sisteme veya veriye zarar vermez

6. Teslim edilecekler (1/2)

#	Teslim	Biçim
1	Kaynak kod	Git deposu (şablondan oluşturulmuş), C++ DLL/.so + SQLite + SoftHSM
2	Birim testleri + CI kaydı	CTest raporu + CI günlüğü
3	Güvenlik kılavuzu (S0–S17)	.docx / .pdf — vizede S0–S5, finalde tam
4	Test sonuçları (S16)	Kılavuz bölümü — yalnız final

6. Teslim edilecekler (2/2)

#	Teslim	Biçim
5	Uyum matrisi (S17) + devredilenler (S14)	Kılavuz bölümü — yalnız final
6	SBOM + bağımlılık taraması	CycloneDX dosyası — yalnız final
7	Sürüm (release)	Git etiketi: <code>midterm-v1.0</code> / <code>final-v1.0</code>
8	Gösterim	Canlı sunum (~10 dk) — vizede 7. hafta, finalde 15. hafta

Güvenlik kılavuzu bölümleri (S0–S17)

S0–S8	S9–S17
S0 Kapak, sürüm geçmişi	S9 Kod sağlamlaştırma (ileri)
S1 Kapsam, kısaltmalar, kaynaklar	S10 RASP + tepki politikası
S2 Ürün ve mimari	S11 Güvenli iletişim (TLS)
S3 Arayüzler	S12 Raporlama/günlükleme
S4 Tehdit modeli (STRIDE, ağaç)	S13 Geliştirme süreci, SBOM
S5 Varlık listesi + koruma	S14 Varsayımlar ve devredilenler
S6 Kimlik/bağlama	S15 Derleme, imzalama, dağıtım
S7 Veri güvenliği + kabuk matrisi	S16 Güvenlik testi ve sonuçları
S8 Kripto, anahtar yaşam döngüsü	S17 Gereksinim uyum matrisi

Tek ZIP arşivi yapısı

```
cen429-vize-ad-soyad.zip
├── cen429-proje-ad-soyad-cpp/
│   ├── lib/
│   ├── app/
│   ├── test/
│   ├── docs/
│   ├── security-guide/
│   ├── sbom/
│   ├── test-coverage/
│   └── README.md
# final: cen429-final-ad-soyad.zip
# GitHub deposunun klonu
# güvenlik kütüphanesi (DLL/.so)
# konsol uygulaması
# CTest birim testleri
# Doxygen çıktısı
# S0-S17 güvenlik kılavuzu
# CycloneDX SBOM (final)
# CTest kapsam raporu
```

Adlandırma

- **Depo:** `cen429-proje-ad-soyad-cpp`
- **Arşiv:** `cen429-vize-ad-soyad.zip` / `cen429-final-ad-soyad.zip`
- **Güvenlik kılavuzu dosyası:** `cen429-guvenlik-kilavuzu-ad-soyad.docx`

Rapor/kılavuzun kapak sayfasında **GitHub deposu bağlantısı** bulunur.

7. Takım çalışması (1/2)

- **GitHub Flow:** `main` dalı korumalıdır, doğrudan commit yapılmaz; her iş için ayrı bir özellik dalı (feature branch) açılır, birleştirme **çekme isteği (pull request)** ile yapılır
- **Commit mesajı kuralı:** [Conventional Commits](#), ör.
`feat(crypto): add AES-GCM wrapper`, `fix(rasp): correct debugger detection`

7. Takım çalışması (2/2)

- **Çekme isteği + inceleme:** her PR en az bir takım arkadaşınca **incelenir (code review)**
- **Issues + Projects panosu:** Backlog → In Progress → In Review → Done
- **CI yeşil olmadan birleştirme yok**
- **Sürüm etiketleri:** vize `midterm-v1.0`, final `final-v1.0`
- **"Bitti" tanımı:** derleniyor · birim testi var · kapsam düşmedi · PR onaylandı · CI yeşil · kılavuz güncellendi

8. Rubrikler

Her kriter 1–5 düzeyinde puanlanır

Puan = (düzey ÷ 5) × kriter puanı

8.1 Başarı düzeyleri

Düzy	Anlamı
5 — Mükemmel	Her şey çalışıyor, test edilmiş ve belgelenmiş
4 — İyi	Küçük eksikler; genel olarak tamam ve test edilmiş
3 — Yeterli	Temel işlemler çalışıyor; belirgin eksikler var
2 — Zayıf	Derleniyor ama işlemlerin çoğu yanlış ya da eksik
1 — Kanıt yok	Teslim edilmemiş ya da çalışmıyor

8.2 RAP1 rubriği (7. hafta, 100 puan)

#	Kriter	ÖÇ	Puan
1	Tehdit modeli ve varlık listesi	ÖÇ.1, ÖÇ.5	15
2	Veri güvenliği: aktarım, bekleme, kullanım	ÖÇ.2, ÖÇ.4	20
3	Kod sağlamlaştırma ve bellek koruması	ÖÇ.3	20
4	RASP ve tepki politikası	ÖÇ.3	10
5	Geliştirme ortamı ve değişiklik yönetimi	ÖÇ.5	10
6	Birim testleri ve CI	ÖÇ.6	10
7	Güvenlik kılavuzu (S0–S5) ve gösterim	ÖÇ.7	15
	Toplam		100

8.3 RAP2 rubriği (15. hafta, 100 puan)

#	Kriter	ÖÇ	Puan
1	Kriptografi ve sertifikalar	ÖÇ.2, ÖÇ.4	15
2	Binary uygulama koruması, gizleme, çeşitlendirme	ÖÇ.3	20
3	Varlık yönetimi ve dinamik varlıklar	ÖÇ.5	10
4	Güvenlik testi ve gözlenen sonuçlar	ÖÇ.6	20
5	Standartlar, uyum matrisi, devredilenler	ÖÇ.7	15
6	OWASP, SBOM ve derleme/dağıtım hattı	ÖÇ.1, ÖÇ.5	10
7	Rapor, sunum ve gösterim	ÖÇ.7	10
	Toplam		100

Her satır bir **kanıt** ister: "karşılandı" diyen satırın kanıtı yoksa karşılanmamış sayılır.

9. Kabul koşulları

Teslim kabul edilmez, eğer...

- GitHub deposu yoksa, **özel (private)** değilse ya da takım üyelerinin commit'i görünmüyorsa
- gereksinimlerin karşılanma oranı asgari eşiğin altındaysa
- depoda ya da arşivde **binary dosyalar** (derlenmiş `.exe` / `.dll` / `.so`) varsa
- sürüm (release) üretilmemişse, uygulama Windows ya da WSL/Linux'ta **derlenmiyor/çalışmıyorsa**
- depoda gerçek sır, anahtar ya da kişisel veri varsa, **intihal** tespit edilmişse

10. Gösterimde sorulacaklar (1/2)

- **Git/GitHub:** Depoyu şablondan doğru adla oluşturdunuz mu? Depo özel mi, ders sorumlusu işbirlikçi mi? Her iki üyenin de commit'i var mı?
- **Kurulum-derleme:** Uygulamayı ve kütüphaneyi Windows'ta ve WSL/Linux'ta derleyin;
`lib / app / test`
ayrımını gösterin

10. Gösterimde sorulacaklar (2/2)

- **Konu (satır satır):** Seçtiğiniz güvenlik gereksinimini kodda satır satır açıklayın
- **Test ve belge:** Birim testlerini ve kapsam raporunu açın; kılavuzun ilgili S bölümünü gösterin
- **Dosya/veri işlemleri:** Bir kayıt ekleyin, programı kapatıp yeniden açın, verinin şifreli geldiğini gösterin
- **Programlama:** Bellek yönetimi, işaretçiler, derleyici/OS korumaları, hata ayıklayıcıda inceleme

11. Mesleki sorumluluk ve akademik dürüstlük

- Teslim ettiğiniz her satırı **açıklayabilmelisiniz**
- Başkasının kodunu/metnini kullandıysanız **kaynağını belirtin**
- Bütün örnekler ve değerler **sentetik** olmalıdır
- **İntihal** ve izinsiz kopyalama başarısızlık nedenidir
- Hayalet commit (başkası adına commit atmak) intihal sayılır

Güvenli ve yasal çerçeve

Sızma testi yalnız **kendi** projeniz üzerinde yapılır.

Örnekler ve değerler **sentetik** olmalı; hiçbir gerçek sır, anahtar ya da kişisel veri depoya girmemeli.

Uygulamanız başkasının sistemine/verisine zarar vermemeli.

12. Sık sorulan sorular (1/2)

Takımı tek başıma kurabilir miyim?

Evet. Takım en çok 5 kişidir; takımlar 3. haftanın sonunda (04.10.2026) kesinleşir.

Konumu onaydan sonra değiştirebilir miyim?

Hayır. Onaydan sonra konu değişmez.

12. Sık sorulan sorular (2/2)

Bir gereksinimi tam karşılayamazsam ne olur?

S14'e gerekçenizi yazarak devredebilir ya da kapsam dışı bırakabilirsiniz.

RAP1'de eksik kalan bir gereksinim RAP2'de tamamlanabilir mi?

Evet, ama RAP1 puanı gösterim anındaki duruma göre verilir ve güncellenmez.

Geç teslim kabul edilir mi?

Hayır — izlencedeki kurallar geçerlidir.

Ek — Proje konuları listesi

100 konu · 4 grup

- Ödeme ve ticaret (01–25)
- Sağlık, eğitim, kamu ve kimlik (26–50)
- Medya, lisans, iletişim ve yazılım tedarik zinciri (51–75)
- IoT, endüstri, ulaşım ve kurum (76–100)

Bütün veriler sentetiktir; sunucu tarafı yalnız kendi bilgisayarınızda (localhost) çalışan bir benzetimdir.

Örnek konu — 01

Çevrimdışı Ödeme Cüzdanı

Kullanıcının internet olmadan küçük ödemeler yapabildiği bir cüzdan. Bakiye ve tek kullanımlık ödeme

anahtarları cihazda durur; bağlantı gelince işlemler sunucuyla eşleşir.

Öne çıkan gereksinimler: 4.4 Beklemedeki veri (AES-GCM + SoftHSM), 4.3 Aktarımdaki veri (TLS 1.3 + sabitleme), 4.10 RASP (hata ayıklayıcı algılanınca anahtar silinir)

Örnek konu — 26

Kişisel Sağlık Kaydı Kasası

Kullanıcının sentetik tanı, ilaç ve alerji kayıtlarını yerel bir kasada tuttuğu uygulama. Saldırgan, cihazı ele geçiren ya da kaybolan cihazı bulan kişidir.

Öne çıkan gereksinimler: 4.4 Beklemedeki veri (SQLite + AES-GCM + SoftHSM), 4.2 Kullanımdaki veri

(sınırlı süre bellekte açık), 4.11 Bellek koruması (görüntülenen kayıt kapatılınca silinir)

Tam liste nerede?

Kalan **98 konu**, her biri aynı biçimde (kısa tanım, temel özellikler, korunacak varlıklar, öne çıkan gereksinimler) **Proje Rehberi** sayfasının sonundaki **Ek — Proje konuları listesi** bölümündedir.

ucoruh.github.io/cen429-secure-programming/project-guide/

Sorular

Proje rehberi: ucoruh.github.io/cen429-secure-programming/project-guide/

Ders sitesi: ucoruh.github.io/cen429-secure-programming